SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This section, "Z Intelligence", introduces the strategic solution for the identified problem of fragmented data and limited analytical capabilities faced by SME e-commerce sellers. It details how a Decision Intelligence (DI) Command Center can unify multi-platform data to optimize economic outcomes.

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decisions behind the Z Intelligence development.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *The system is a cloud-native platform designed around a multi-agent orchestration layer using the CrewAI framework.*
    b. *At its core, the architecture features a centralized Intelligence Hub powered by Z.AI's ilmu-glm-5.1.*
    c. *The backend is modularly structured into a Data Integration Service (Shopee/Lazada/TikTok APIs), a Pre-aggregation Engine (arithmetic processing), and an AI Reasoning Layer. This modularity ensures that while data ingestion is distributed, the decision-making logic is unified and robust.*
2. ***Data Flows:***
    a. *Fragmented e-commerce data (Orders, Inventory, and Ads) is acquired via Shopee, Lazada, TikTok Shop,and other E-commerce platforms APIs.*
    b. *This raw "scattered data" is transformed into high-density JSON summarized by the Pre-aggregation Engine where formulas (like Net Profit and Margin %) are calculated.*
    c. *These summaries are then injected into the ilmu-glm-5.1 context window by the Strategist Agent.*
    d. *The model interprets these financial signals to generate localized, actionable growth strategies, which are then pushed to the user's real-time web dashboard.*
3. ***Model Process:***
    a. *The system follows an end-to-end workflow driven by ilmu-glm-5.1's logical deduction capabilities.*
    b. *The process begins with the **Risk Sentinel** scanning for background alerts like stockouts or margin drops.*
    c. *When a risk is identified, the **Strategist Agent** utilizes the GLM to perform a trade-off analysis (e.g., "What-if" pricing) to propose a solution.*
    d. *The process concludes with the Executor Agent generating a professional email or WhatsApp draft, which the human can review and edit before copying for use.*

© UMHackathon 2026

4. ***Role of Reference:***
    a. *[This document serves as a primary technical reference for all the developers and testers to ensure the multi-agent architecture and the integration of ilmu-glm-5.1 are implemented consistently.*
    b. *It defines the clear API usage contracts and database schemas necessary to maintain the system's focus on Decision Intelligence and economic empowerment.*

1.2. ***Background****:*

Small and medium e-commerce sellers currently struggle with "scattered data" across multiple selling and marketing platforms, leading to manual or intuition-based processes.

1. *Previous Version:*
   Most target users lack a unified platform for data analysis and must manually check individual seller centers for different E-Commerce platforms. Available solutions (eg. Domo) is expensive and does not integrate AI as their main tool to help non-technical SME owners.

2. *Changes in Major Architectural Components*
   Moving from manual data tracking to an automated, API-driven command center, and allowing scattered data to be unified for making informed and smart business decisions.

3. *New capabilities that are only introduced in this*
   Introduction of context-aware reasoning and predictive modeling using a 3-agent GLM architecture to help owners make more revenue.

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
E-commerce SME Seller	*Uses the command center to view unified data and get strategy recommendations.*	*One-click data transfer, increased revenue and simplified decision-making.*
Development Team	*Build the multi-agent framework and maintain the pre-aggregation backend.*	*Modular code, clear API contracts and stable GLM reasoning.*
Strategy Analyst (AI Agent)	*Processes acquired data into meaningful recommendations.*	*Immediate feedback and crystal-clear business status reporting.*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web-based Decision Intelligence (DI) Command Center.*
Architecture	*Guided Multi-Agent "Agent-as-a-Tool" Framework.*
Model	*Centralized Intelligence Hub via Z.AI ilmu-glm-5.1*

Z Intelligence is primarily structured as a client-server-based system in which there is one central client: a Web-based Decision Intelligence Dashboard designed for e-commerce SME sellers. This client communicates with a Backend API layer that orchestrates three independent services: the Data Integration Service (handling Shopee, Lazada, and TikTok Shop data), the Pre-aggregation Engine (processing mathematical calculations), and the Multi-Agent Orchestration Service (managing the Risk Sentinel, Strategist, and Executor agents).

The system utilizes a three-agent GLM architecture to transform scattered data into meaningful information and recommendations. All services are deployed on a cloud-native platform, utilizing a Firebase Firestore to ensure persistent storage of synchronized order, product, and inventory data. To maintain arithmetic integrity, the backend pre-calculates all KPIs before passing summarized insights to the GLM as a service layer.

2.1.2 LLM as Service Layer

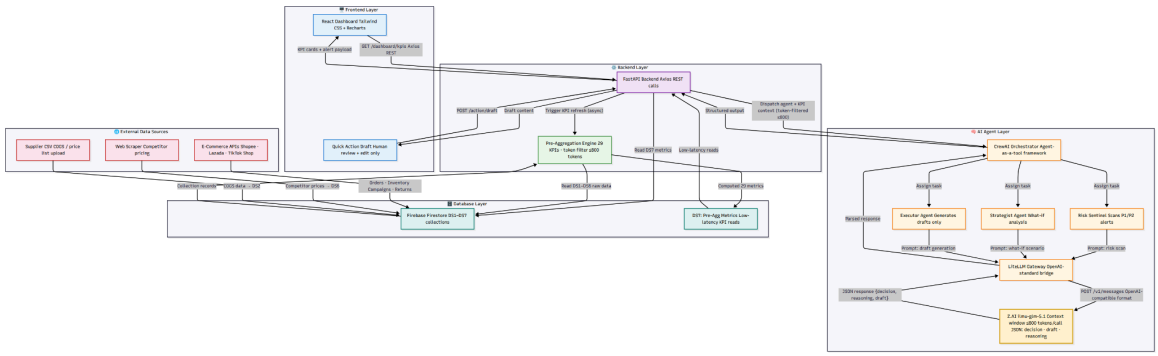
Rather than a generic "AI Module," ilmu-glm-5.1 is integrated as the system's "Reasoning Layer."

- ***Prompt Construction:*** *Prompts are dynamically generated using* ***Structural Templates.*** *The backend injects system instructions (e.g., "Always use RM for currency") combined with a* ***JSON-formatted snapshot*** *of the seller's real-time performance data.*
- ***Context Window Management:*** *To ensure high-quality output, the context window is populated with three specific data tiers:*
    1. *Seller Profile: Historical performance bounds.*
    2. *Aggregated KPIs: Pre-calculated summaries from the Pre-aggregation Engine.*
    3. *External Intel: Scraped competitor pricing and regional sale event data (11.11, 12.12).*

© UMHackathon 2026

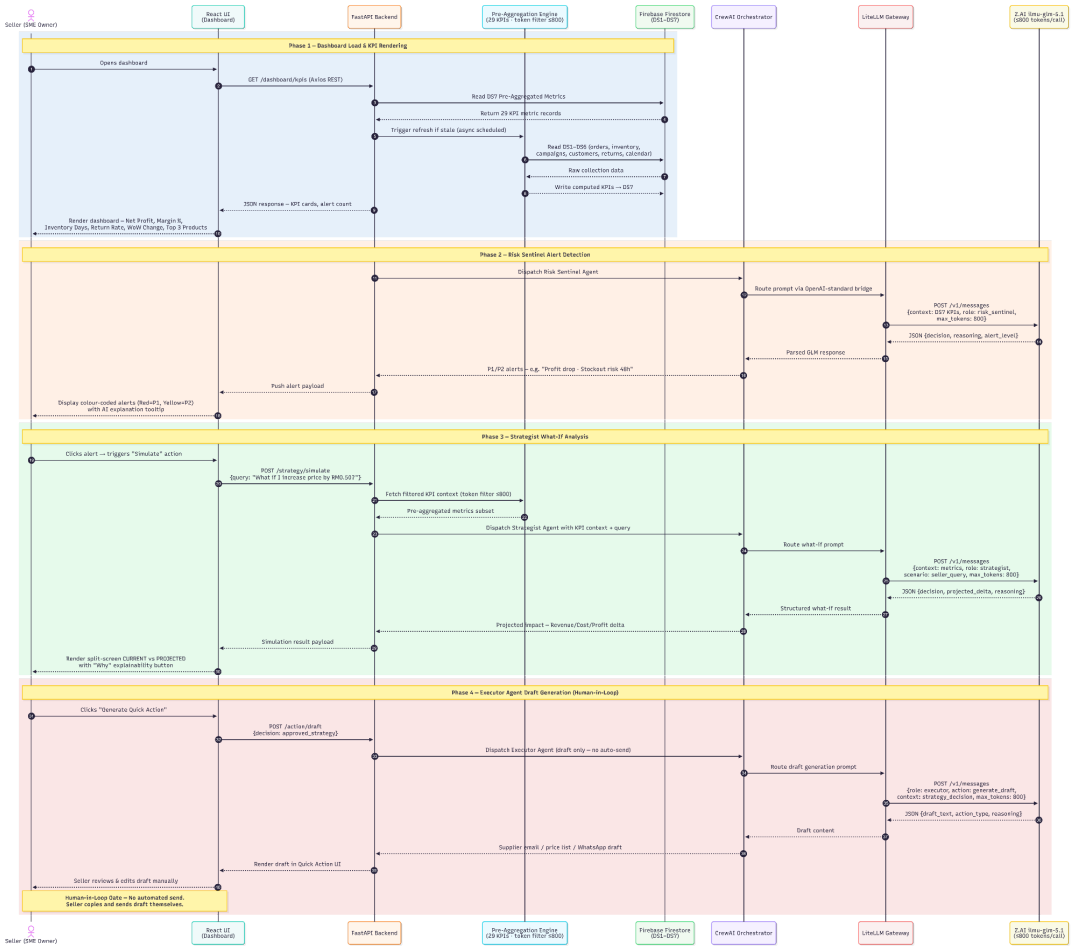
- ***Token Enforcement & Chunking:*** *Before data reaches the GLM, the* ***Pre-aggregation Engine*** *acts as a "token filter." Instead of passing thousands of raw order lines, it passes a summarized array of the Top 5 performing/underperforming items. The context is strictly capped at* ***800 tokens per agent call*** *to prevent reasoning drift.*
- ***Response Parsing:*** *The system enforces a* ***Structured Output*** *contract. The GLM is instructed to return responses in JSON. The Backend parses these objects to distribute data:*
    - *Decision field → Dashboard "Decision of the Day."*
    - *Draft field → Quick Actions component.*
    - *Reasoning field → Insight "AI Explanation" box.*

2.1.3 Dependency Diagram



Google drive link of Dependency Diagram:
https://drive.google.com/file/d/1cYYwH5extJ8WSwr5PL3sYAbBBxuLhK0C/view?usp=sharing

2.2.2. Sequence Diagram



Google drive link of Sequence Diagram:
https://drive.google.com/file/d/16vdZngnM30o5t9eOM1uYvVKPoaoRXOQF/view?usp=sharing

2.2. Technological Stack

2.2.1. Frontend

React – Utilizing Tailwind CSS for a responsive, high-density dashboard and Recharts for data visualization.

2.2.2. Backend

FastAPI – High-performance RESTful framework for handling asynchronous database queries and agent orchestration.

2.2.3. Database

Firebase Firestore – NoSQL cloud database for real-time synchronization for multi-platform e-commerce datasets.

2.2.4. AI Engine

© UMHackathon 2026

Z.AI ilmu-glm-5.1 – Orchestrated via CrewAI and LiteLLM.

2.2.5. Cloud/Deployment

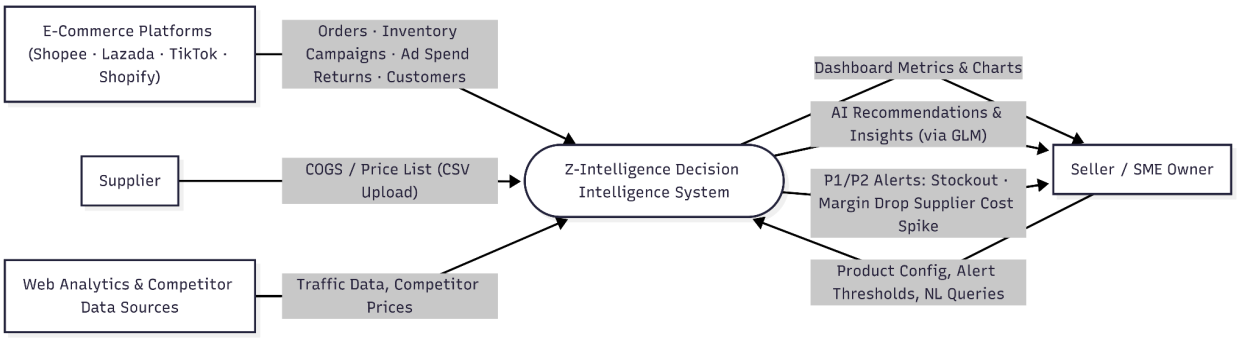
Vercel (Frontend) and Cloud-hosted Virtual Machine (Backend) to ensure 24/7 availability for the Risk Sentinel monitor.

2.3. Key Data Flows

2.3.1. This section shows how data is moving through the system and how it's structured and stored.

2.3.1.1. Data Flow Diagram (DFD)

a. The Level 0 DFD shows Z Intelligence as a single system process, illustrating all external entities and the data flows entering and leaving the system boundary.

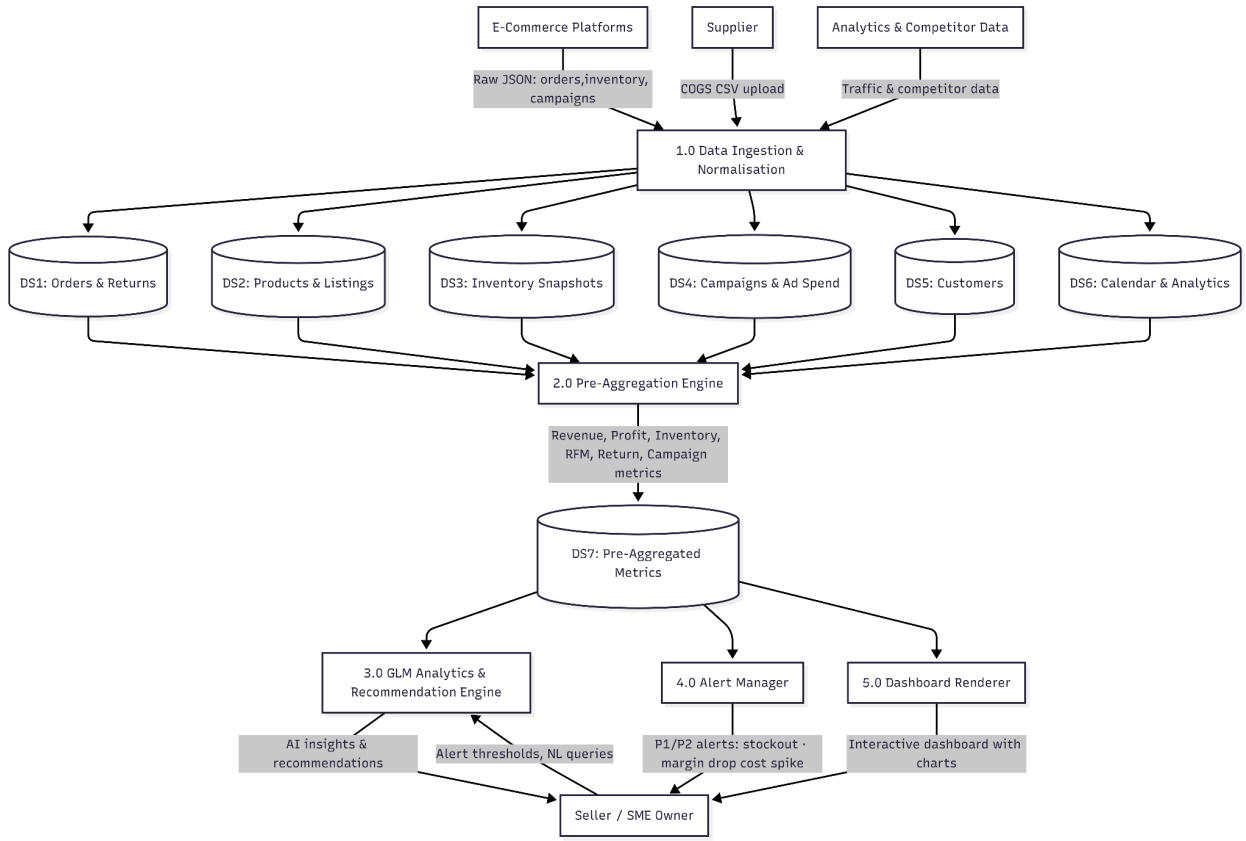


b. External Entities

Entity	Who They Are	Data Sent to Z Intelligence	Data Source Method
E1: E-Commerce Platforms	Shopee, Lazada, TikTok Shop, Shopify	Orders, Inventory, Campaigns, Ad Spend, Returns, Customers	Platform Orders & Inventory APIs
E2: Seller / SME Owner	Primary user — the business operator	Product info, alert thresholds, natural language queries	Human input via UI
E3: Supplier	Product cost provider	COGS/price list updates	CSV file upload
E4: Analytics & Competitor Sources	Third-party data providers	Web traffic data, competitor product prices	Analytics APIs, web scrapers

© UMHackathon 2026

c. *The Level 1 DFD decomposes Z Intelligence into 5 major processes, 7 data stores, and the complete data flows between them. This diagram satisfies the DFD Black Hole rule — every process receives and produces data.*



d. *Process Descriptions*

Process	Description	Inputs	Outputs
1.0 Data Ingestion & Normalisation	Ingests raw JSON/CSV from all platform APIs and supplier uploads. Validates, deduplicates, transforms to 3NF schema. Enforces referential integrity between tables.	All external entities	DS1–DS6 data stores
2.0 Pre-Aggregation Engine	Computes all 29 metrics from raw data (see Section 6). Runs on schedule (daily/hourly) or on-demand. Outputs pre-computed metrics to DS7 for low-latency dashboard queries.	DS1–DS6	DS7: Pre-Aggregated Metrics
3.0 GLM Analytics & Recommendation Engine	Passes pre-aggregated metrics + seller query as structured prompt to Z.AI GLM. Receives context-aware insights,	DS7 + Seller queries	Seller (AI output)

© UMHackathon 2026

Process	Description	Inputs	Outputs
	recommended actions, and explanations in plain language.		
4.0 Alert Manager	Monitors DS7 for threshold breaches: stockout risk (≤7 days), margin drop (>10%), COGS spike (>10%), return rate surge. Classifies as P1 (critical) or P2 (warning). Pushes alerts to seller.	DS7	Seller (alert output)
5.0 Dashboard Renderer	Reads from DS7 to render the 6 primary KPI cards + per-category charts. Applies Calendar flags for contextual annotations (e.g. 'Today is 11.11'). Serves interactive UI to sellers.	DS7	Seller (dashboard UI)

e. Data Stores

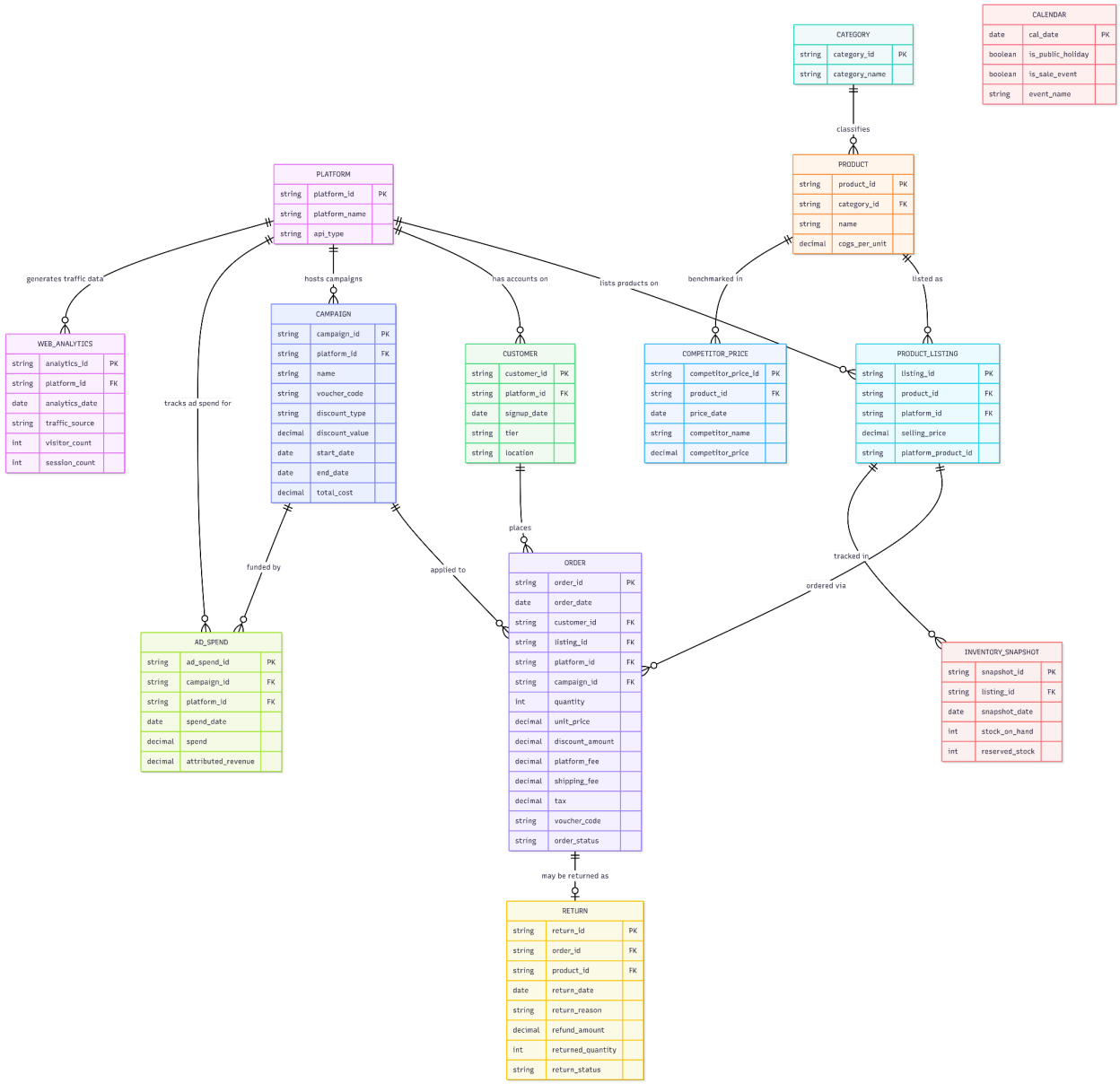
ID	Name	Tables Contained	Populated By
DS1	Orders & Returns	ORDER, RETURN	Shopee/Lazada/TikTok/Shopify + Returns API
DS2	Products & Listings	PRODUCT, PRODUCT_LISTING, CATEGORY	Product API + File Upload
DS3	Inventory Snapshots	INVENTORY_SNAPSHOT	Inventory API (daily)
DS4	Campaigns & Ad Spend	CAMPAIGN, AD_SPEND	Campaign API + Ad Spend API
DS5	Customers	CUSTOMER	Customer API
DS6	Calendar & Analytics	CALENDAR, WEB_ANALYTICS, COMPETITOR_PRICE	Static CSV + Analytics API + Web Scraper
DS7	Pre-Aggregated Metrics	All 29 computed metrics (see Section 6)	Output of Process 2.0

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3.2. Normalized Database Schema

a. The Entity-Relationship Diagram below shows all 13 tables normalized to Third Normal Form (3NF)。



© UMHackathon 2026

Email: umhackathon@um.edu.my

b. 3NF Compliance Verification

3NF Rule	How This Schema Satisfies It	Status
All non-key fields depend on the full primary key only (no partial dependencies)	All tables use single-column surrogate PKs or natural PKs (Calendar uses cal_date). No partial dependencies possible.	✓ 2NF
No transitive dependencies (non-key field determines another non-key field)	category_name extracted to Category table. selling_price moved to ProductListing. stock_on_hand moved to Inventory Snapshot. Platform name in Platform table.	✓ 3NF
M:N relationships resolved	Order–Product M:N resolved via ProductListing (a product can appear on multiple platforms, an order references one listing). Campaign–Platform M:N resolved via platform_id FK on Campaign.	✓ No M:N
No repeating groups	Each field stores one atomic value. platform_id is a FK, not a multi-value field — multiple platforms handled via separate rows in ProductListing.	✓ 1NF

3. *Functional Requirements & Scope*

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

© UMHackathon 2026

Email: umhackathon@um.edu.my

3.1. Minimum Viable Product:

3.1.1. *Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.*

#	Features	Description
1	Risk Sentinel (Agent 1)	A background monitor that automatically scans cross-platform data to detect anomalies like margin drops, stockouts, or sudden return rate spikes. It ranks alerts by severity (P1/P2) so the owner can focus on critical business threats.
2	Strategist Agent (Agent 2)	An on-demand reasoning engine that performs "What-if" simulations, such as projecting the revenue impact of a 10% price increase. It also conducts market research and competitor price analysis to provide actionable recommendations.
3	Executor Agent (Agent 3)	A task-oriented agent that takes approved decisions and transforms them into structured drafts. It generates supplier emails or WhatsApp message drafts that the human can review and edit.
4	One-Click Data Sync	A seamless integration layer that uses mock JSON data to simulate the retrieval of scattered information from Shopee, Lazada, and TikTok Shop APIs. This eliminates manual entry and places all acquired data in one spot.
5	Decision Intelligence Dashboard	A centralized visual interface that leads with the top 6 critical metrics, including Net Profit, Net Margin %, and Inventory Days remaining. It uses pre-aggregated backend data to ensure all displayed KPIs are mathematically accurate.

© UMHackathon 2026

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality	***Requirements***	***Implementation***
Scalability	*Z-Intelligence system must handle increasing data volumes as business owners grow.*	*Architect the data flow to support multi-tenancy and high data volumes, though not the primary focus for the hackathon demo.*
Reliability	*Prevent accuracy drops caused by context window degradation.*	*Deliberately split GLM calls into focused prompt types (Dashboard, Insight, Scenario) to keep each call under 800 tokens.*
Maintainability	*Each modular service (Data Integration Service, Pre-aggregation Engine, and Multi-Agent Orchestration Service) is independently structured so that changes to one component do not affect the others.*	*Each service is developed as a separate module with its own environment configuration, enabling individual updates and testing without disrupting the rest of the system.*
Token Latency	*Insight generation and agent responses must return within 3 seconds.*	*Utilize asynchronous API calls to the GLM and implement a user-facing loading shimmer/state while awaiting responses.*
Cost Efficiency	*Manage the average cost of tokens per user session.*	*Fix prompt lengths, truncate/chunk inputs before they reach the GLM, and cache repeating queries for up to 24 hours.*

© UMHackathon 2026

3.3. Out of Scope / Future Enhancements

a. *Live production API integration with Shopee, Lazada, and TikTok Shop (currently using mock data for the demo).*
b. *Real-time competitive price scraping and automated price adjustment recommendations.*
c. *Hyper-personalized customer loyalty programs and automated promotional voucher generation.*
d. *Automated execution of supplier emails and WhatsApp messages through direct platform integration.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Grayscale Rollout & A/B Testing****:*

New AI reasoning prompts or strategy modules are initially released to 5% of SME sellers. The development team monitors system logs for token latency and response accuracy. If no anomalies are detected, the rollout is gradually increased until it reaches 100% of the user base.

4.1.2. ***Emergency Rollback (ER) & Golden Release****:*

If the automated monitoring system detects a P1 Critical error, such as a platform API connection failure or GLM latency exceeding the 5-second threshold, an ER is triggered immediately. The automated pipeline will immediately redeploy the Golden Version of the application—a stable, verified build that utilizes cached data and basic heuristic-based recommendations to maintain core dashboard visibility while the AI service is restored.

4.1.3. Priority Matrix:

4.1.3.1. *P1 Critical: If platform API sync failure or >1% strategy generation errors, trigger Emergency Rollback to Golden Release and switch to manual data upload mode.*

4.1.3.2. *P2 High: If GLM latency exceeds 3 seconds for >5% of requests, throttle non-essential background agent tasks (e.g., Risk Sentinel) to prioritize the Strategist Agent.*

4.1.3.3. *P3 Medium: If minor UI rendering issues or delay in non-critical metric updates, log an incident for the next maintenance sprint; no immediate rollback required.*

© UMHackathon 2026

4.2. ***Monitoring***:

4.2.1. *The monitoring plan utilizes real-time tracking of platform API connectivity, GLM response latency, and arithmetic integrity to trigger automated fail-safes if system health degrades*

4.2.2. *AI Processing Log: The system maintains a real-time log that records every GLM interaction, including prompt timestamps and token consumption, which serves as the primary diagnostic tool for the development team.*

4.2.3. *Automated Threshold Triggers: The monitoring service is programmed to trigger a P1 Critical state if the platform API connection fails or if the Strategist Agent's response accuracy drops below a verified baseline.*

4.2.4. *Performance Dashboards: A dedicated internal view monitors "Agent Health," tracking the latency of the Risk Sentinel, Strategist, and Executor agents to ensure the user experience remains seamless.*

4.3. ***Assumptions:***

a. *Third-party e-commerce platforms (Shopee, Lazada, TikTok Shop) provide stable API access, and mock JSON data accurately reflects real-world API structures for the demonstration phase.*

b. *SME owners have a stable internet connection to ensure real-time synchronization between the e-commerce platforms and the Z Intelligence dashboard.*

c. *The underlying data provided by the seller (such as COGS per unit or supplier price lists) is accurate and updated regularly to ensure valid AI reasoning.*

d. *Business owners have access to a desktop or tablet device to monitor the decision intelligence dashboard and approve generated strategies.*

e. *The Z.AI GLM service maintains consistent uptime to process context-aware reasoning and strategy generation.*

© UMHackathon 2026

4.4. *External Dependencies:*

Tools	*Purpose*	*Risks*
Z Ai GLM API	*Serves as the core LLM reasoning engine to handle prompt inference, multi-step reasoning, and generating structured strategy outputs.*	*High: In case of GLM API rate limitation, the core product will fail. Mitigation: Implement a graceful degradation state where the system shows pre-aggregated data only.*
E-Commerce Platforms APIs	*Used to transfer data directly to the platform for unified viewing of scattered data.*	*Medium: Token expiry or platform downtime. Mitigation: Use mock JSON data to simulate the API return for the hackathon demo.*
CrewAI Framework	*Orchestrates the multi-agent system (Strategist, Risk Sentinel, and Executor) to manage complex data flows.*	*Low: Framework version instability. Mitigation: Pin specific library versions and conduct rigorous local testing.*
WhatsApp / Email SMTP API	*Allows the Executor Agent to create drafts of supplier emails and WhatsApp messages.*	*Medium: Message delivery failure or spam filtering. Mitigation: Provide an editable draft for the user to copy/paste.*
Competitor Web Scraper	*To obtain competitor names and prices for market research and competitive analysis.*	*Medium: Website structure changes blocking the scraper. Mitigation: Use third-party APIs as a fallback for competitor data.*

© UMHackathon 2026

5. *Project Management & Team Contributions*

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. *Project Timeline:*

- ***Day 1-2:*** *Problem Validation & Team Setup Validated the "Decision Intelligence" narrative to address SME pain points, set up the GitHub repository and completed UI wireframing in Figma.*
- ***Day 3-4:*** *UI/UX Development. Architecture, Data Schema & Pre-aggregation Engine Designed the high-level 3-agent architecture (Risk Sentinel, Strategist, Executor), structured the Firebase Firestore schema and collections, and built the backend logic to pre-calculate the Top 6 Dashboard Metrics (Net Profit, Margin %, etc.) to avoid GLM arithmetic errors..*
- ***Day 5-7:*** *Multi-Agent Logic & Prompt Engineering Integrated CrewAI and LiteLLM to orchestrate the Z.AI ilmu-glm-5.1 model across agent interactions, and optimized prompt categories (under 800 tokens) for Revenue, Inventory, and Risk alerts.*
- ***Day 8-9:*** *"Quick Actions" Integration Built the React + Tailwind CSS dashboard interface with Recharts data visualizations, and implemented the Quick Action Smart Draft generator for the Executor Agent, producing structured email/WhatsApp drafts that the human can review and edit*
- ***Day 10:*** *Testing, Evaluation & Final Pitch Prep Conducted simulated scenario testing, finalized the mock data demo, completed all submission documents (PRD, SAD, QATD), and prepared the Pitch Deck and 10-minute pitching video for judges.*

5.2. *Team Members Role:*

- *Sam Hui Ying - Role: Project Lead & Documentation Lead: - Oversees project planning, milestone tracking, and ensures the system delivers measurable economic outcomes like time saved and revenue improvement. Complete all the documents.*

- *Khong Yirou - Role: Backend Lead & Data Architect - Plans and builds solutions' System Architecture, Data Structure, and flow. Responsible for data manipulation and integration, ensuring smooth parsing of data across the website, database, and GLM.*

- *Tang Yvone - Role: AI & Prompt Engineer - Manages the CrewAI framework, agent-as-a-tool hierarchy, and the integration of Z.AI ilmu-glm-5.1 with the Firestore database. Responsible for prompt optimization to prevent context window degradation.*

© UMHackathon 2026

- *Chong Yu En - Role: UI & UX Designer - Designs and develops the overall UI/UX for Z Intelligence, including the dashboard, data extraction page, insights module, and quick action features. Responsible for crafting intuitive user flows, wireframes, and high-fidelity prototypes.*

- *Oi Kay Yi - Role: Frontend & UX Developer - Responsible for the technical implementation of the overall frontend design. Translated Figma mockups into a functional React interface, specifically building the interactive charts, the animated AI Processing Log, and the structured "Quick Action Smart Draft" components.*

5.3. Recommendations:

- ***Prompt Caching & Versioning:*** *Implement dedicated prompt management tools to version and cache common strategy reasoning patterns. By using Redis to cache responses for identical KPI snapshots, the system can ensure consistent agent behavior and significantly lower operational token costs per session.*
- ***Asynchronous Event-Driven Sync:*** *Move toward an asynchronous architecture using message brokers (e.g., RabbitMQ or Google Cloud Pub/Sub) for platform data ingestion. This ensures that heavy background syncs from Shopee or Lazada do not block the UI, maintaining a snappy "Command Center" experience for the user.*
- ***Multi-Tenant Data Partitioning:*** *As the platform scales to support thousands of SME sellers, implement multi-tenant database partitioning. This ensures strict data isolation for security and maintains high query performance by preventing "noisy neighbor" issues in large historical datasets.*
- ***Heuristic-Based "Safe Mode":*** *Maintain a library of heuristic-based "Local Logic" recommendations. If the ilmu-glm-5.1 service experiences downtime or high latency, the system should automatically trigger these rule-based insights, ensuring the merchant is never left without basic guidance.*
- ***Agentic State Management:*** *Implement a state-management framework (like LangGraph) to allow agents to "handoff" tasks more dynamically. This would enable the Risk Sentinel to not only alert but also autonomously trigger the Strategist to begin a simulation before the user even opens the dashboard, further reducing the time-to-insight.*

© UMHackathon 2026